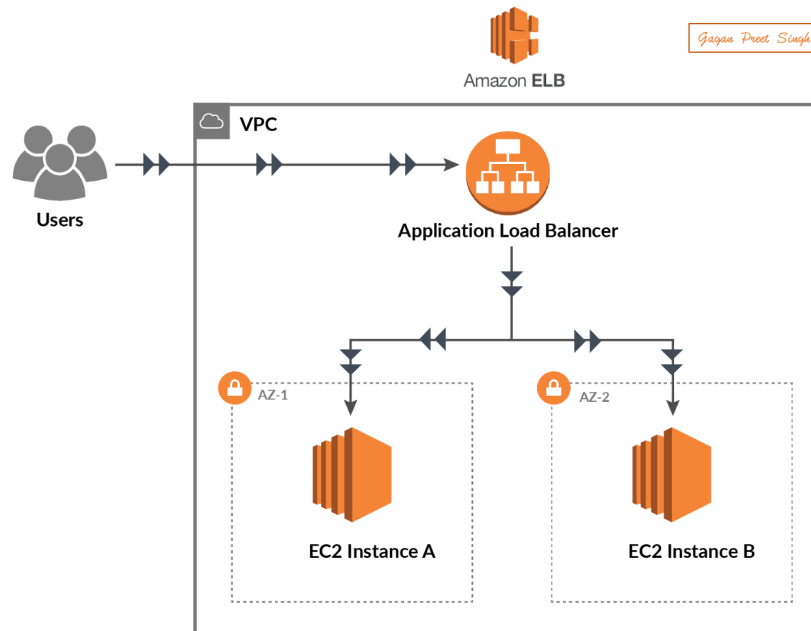


Optimizing Web App High Availability with AWS ALB and EC2:



Setting up a basic two-tier web application where an ELB distributes incoming traffic across multiple EC2 instances.

What is Amazon Application Load Balancing?

Application Load Balancing distributes incoming traffic across multiple servers to improve performance, reliability, and scalability. It enhances user experience by ensuring efficient handling of requests and preventing server overload.

How I used Amazon ALB in this project?

Configured an AWS Application Load Balancer (ALB) with an HTTP listener to distribute incoming HTTP traffic from users across multiple EC2 instances (virtual servers). The ALB operates at Layer 7 of the OSI model, allowing for advanced routing and filtering of requests based on HTTP headers and other attributes. To ensure optimal performance and reliability, health checks were implemented to monitor the health of target EC2 instances. Unhealthy instances are automatically removed from the load balancer's rotation, preventing them from receiving traffic and impacting user experience.

Challenges:

One unexpected challenge I encountered was the complexity of configuring security groups for my EC2 instances. To ensure optimal security, I implemented a layered approach:

Security Group Configuration:

- **Inbound Rules:**
 - I restricted inbound traffic to only essential ports:
 - **HTTP (Port 80):** Allowed incoming web traffic to reach the EC2 instances.
 - **SSH (Port 22):** Enabled secure remote access for administrative purposes.
- **Outbound Rules:**
 - Outbound traffic is set to allow all destinations by default. This ensures the instances have unrestricted access to external services required for application functionality, such as updates and API calls.

Public Key Authentication:

To further enhance security, I implemented public key authentication:

1. **Key Generation:** I generated a pair of public and private keys for each authorized user.

2. **Public Key Distribution:** The public key was securely added to the authorized-keys file on the EC2 instances.
3. **Private Key Security:** The private key was kept confidential by each user, ensuring that only authorized individuals could access the instances.

Learning about crucial network security strategies allowed me to maintain and optimize services along the way to ensure EC2 infrastructure was reliable, robust and secured.

Configuration and Setup:

Launching an EC2 Instance:

To set up the EC2 instance for this project, I navigated to the **EC2 Dashboard** in the AWS Management Console and selected **Launch Instance**. I chose **Amazon Linux** as the **Amazon Machine Image (AMI)**, set the instance type to **t2.micro**, and configured network settings by creating security groups for **SSH (port 22)** and **HTTP (port 80)**. Additionally, I set up storage options and created a **key pair** to secure access. After reviewing all settings, I finalized the setup by clicking **Launch Instance**, successfully deploying the EC2 instance for the project.

Importance of Configuring Inbound Traffic:

Configuring inbound traffic rules is essential to control access to the instance and ensure security. Allowing **SSH (port 22)** is necessary for secure remote access to the server, enabling me to connect via a secure terminal for administration and configuration. Enabling **HTTP (port 80)** allows incoming web traffic, making it possible to host and serve web applications. Without these settings, the instance would be inaccessible from the internet, limiting its functionality and utility for the project.

The screenshot displays the AWS Management Console interface for creating a new EC2 instance. The 'Security group' section is expanded, showing two inbound rules: 'Security group rule 1' for SSH (port 22) and 'Security group rule 2' for HTTP (port 80). The 'Summary' section on the right shows the instance configuration: 1 instance, Amazon Linux 2023 AMI, t2.micro instance type, new security group, and 1 volume of 8 GiB. A 'Free tier' notification is also visible.

Adding User Data Script to Install Apache

To configure the instance for web hosting upon launch, I added a **User Data** script that installs and starts the Apache web server automatically. This script runs upon the first boot of the instance and performs the following steps:

1. Install Apache:

- Updates the system packages.
- Installs Apache.
- Starts and enables the Apache service to run at boot.

```
#!/bin/bash
sudo yum update -y
sudo yum install httpd -y
sudo systemctl start httpd
sudo systemctl enable httpd
```

2. Create a Simple Web Page:

- Adds a basic HTML file to the default web directory, which displays a unique message to identify each instance.

```
echo "Hello from EC2 Instance 1" | sudo tee /var/www/html/index.html
```

This setup ensures that when the instance is launched, it is ready to serve a web page immediately without any additional configuration.

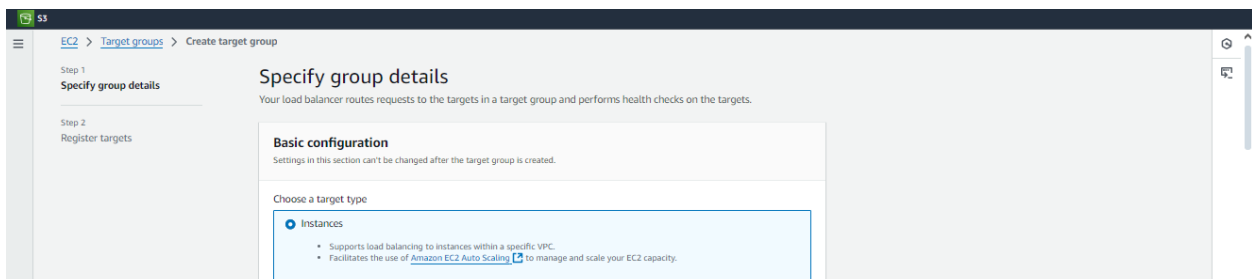
Instances:

☐	web-applicati...	i-0f3be713a317f9469	Running	Q Q	t2.micro	2/2 checks passed	View alarms +	ap-south-1a	ec2-13-234-112-116.ap...	13.234.112.116
☐	web-applicati...	i-0fd654e72a349c94b	Running	Q Q	t2.micro	2/2 checks passed	View alarms +	ap-south-1a	ec2-13-233-195-188.ap...	13.233.195.188

Create Target Groups:

I created two target groups to ensure optimal performance and reliability for my application. One target group was assigned to EC2 Instance A, and the other to EC2 Instance B. By configuring health checks for each group, I enabled the Application Load Balancer to distribute traffic evenly across both instances, while also routing traffic only to healthy ones. This setup enhances the overall system's availability.

Specified the target groups in this project target groups were EC2 instances:



Target group name

tg-01

A maximum of 32 alphanumeric characters including hyphens are allowed, but the name must not begin or end with a hyphen.

Protocol : Port

Choose a protocol for your target group that corresponds to the Load Balancer type that will route traffic to it. Some protocols now include anomaly detection for the targets and you can set mitigation options once your target group is created. This choice cannot be changed after creation

HTTP 80

1-65535

IP address type

Only targets with the indicated IP address type can be registered to this target group.

☒ IPv4

Each instance has a default network interface (eth0) that is assigned the primary private IPv4 address. The instance's primary private IPv4 address is the one that will be applied to the target.

☐ IPv6

Each instance you register must have an assigned primary IPv6 address. This is configured on the instance's default network interface (eth0). [Learn more](#)

VPC

Select the VPC with the instances that you want to include in the target group. Only VPCs that support the IP address type selected above are available in this list.

vpc-0cb56cae882200c2

IPv4 VPC CIDR: 172.31.0.0/16

Protocol version

☒ HTTP1

Send requests to targets using HTTP/1.1. Supported when the request protocol is HTTP/1.1 or HTTP/2.

☐ HTTP2

Send requests to targets using HTTP/2. Supported when the request protocol is HTTP/2 or HTTPS.

I configured the target group to use HTTP on port 80, ensuring compatibility with my web application. I selected IPv4 as the IP address type and specified the VPC where my EC2 instances reside. By choosing HTTP/1.1 as the protocol version, I optimized the communication between the load balancer and target instances.

Register the target with the target groups which enables ALB to route traffic to the target groups

0 selected

Ports for the selected instances

Ports for routing traffic to the selected instances.

80

1-65535 (separate multiple ports with comma)

Include as pending below

2 selections are now pending below. Include more or register targets when ready.

Review targets

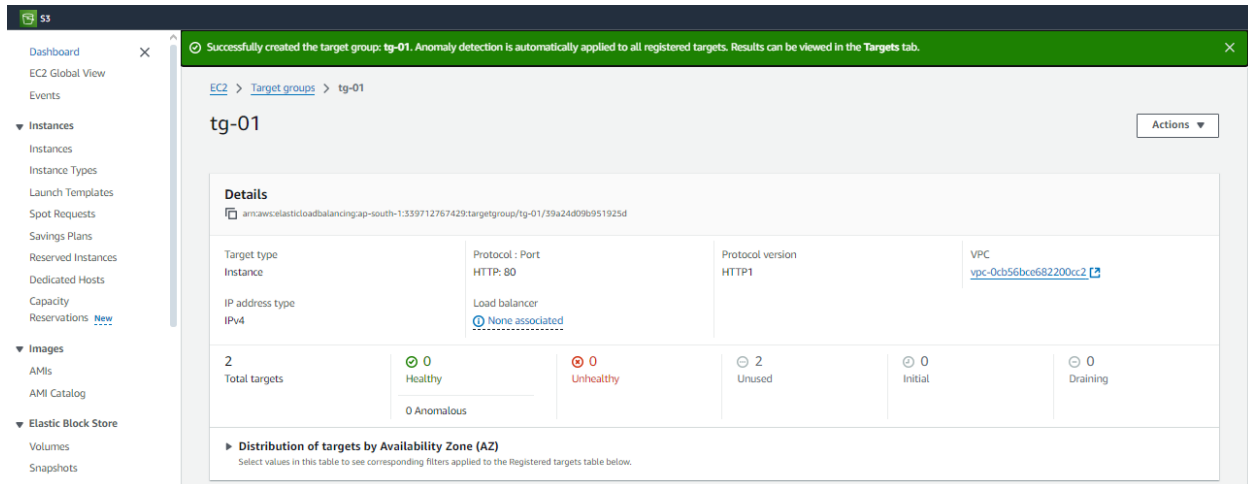
Targets (2)

Remove all pending

Filter targets

Show only pending

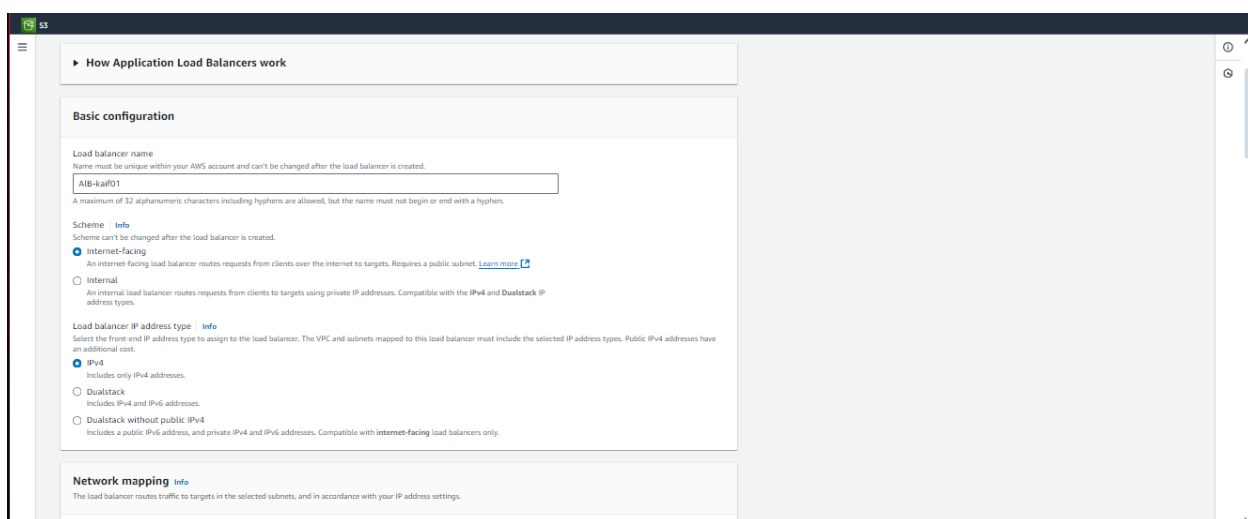
Instance ID	Name	Port	State	Security groups	Zone	Private IPv4 address	Subnet ID	Launch time
i-0f654e72a349c94b	web-application-instance-B	80	Running	alb-instanceB-sg1	ap-south-1a	172.31.46.252	subnet-03e61c4dbb4c2d596	November 12, 2024, 04:59 (UTC+5:30)
i-0f5be713a317f9469	web-application-kaifproject	80	Running	alb-sg-1	ap-south-1a	172.31.42.19	subnet-03e61c4dbb4c2d596	November 12, 2024, 04:49 (UTC+5:30)



Target group is configured for HTTP on port 80 in VPC . By registering EC2 instances within these groups, Creating the Application Load Balancer will help to distribute traffic efficiently, improving performance and reliability.

Create Application Load Balancer (ALB):

The following step is followed to create Application Load Balancer ALB. Basic configuration requires ALB name, Internet-facing scheme for handling request from clients over the internet, type of IP address for load balancer to handle those ipv4 addresses.



Next step requires network mapping configuration which requires selecting Virtual Private Cloud VPC, subnets essential for public access over the internet and Availability Zones AZ in different regions which ensure EC2 instances are up and running without any failure.

Network mapping [info](#)

The load balancer routes traffic to targets in the selected subnets, and in accordance with your IP address settings.

VPC [info](#)

The load balancer will exist and scale within the selected VPC. The selected VPC is also where the load balancer targets must be hosted unless routing to Lambda or on-premises targets, or if using VPC peering. To confirm the VPC for your targets, view [target groups](#). For a new VPC, [create a VPC](#).

vpc-0c5b5bce682200xc2
IPv4 VPC CIDR: 172.31.0.0/16

Mappings [info](#)

Select at least two Availability Zones and one subnet per zone. The load balancer routes traffic to targets in these Availability Zones only. Availability Zones that are not supported by the load balancer or the VPC are not available for selection.

Availability Zones

☒ **ap-south-1a (aps1-az1)**

Subnet
subnet-03e61c4dbb4c2d596
IPv4 subnet CIDR: 172.31.32.0/20

IPv4 address
Assigned by AWS

☒ **ap-south-1b (aps1-az3)**

Subnet
subnet-01946b96de4b37510
IPv4 subnet CIDR: 172.31.0.0/20

IPv4 address
Assigned by AWS

Last step for creating Application Load Balancer ALB requires configuration of listener rules which enables to sent those HTTP requests to the registered target groups.

Listeners and routing [info](#)

A listener is a process that checks for connection requests using the port and protocol you configure. The rules that you define for a listener determine how the load balancer routes requests to its registered targets.

Listener HTTP:80 [Remove](#)

Protocol: HTTP Port: 80
1-65535

Default action [info](#)

Forward to: tg-01 HTTP
Target type: Instance, IPv4
[Create target group](#)

Listener tags - optional

Consider adding tags to your listener. Tags enable you to categorize your AWS resources so you can more easily manage them.

[Add listener tag](#)

You can add up to 50 more tags.

[Add listener](#)

The following step shows the active state of the Application Load Balancer ALB along with the 3 Availability Zones.

EC2 > Load balancers

Load balancers (1) Actions Create load balancer

Elastic Load Balancing scales your load balancer capacity automatically in response to changes in incoming traffic.

Filter load balancers

☐	Name	DNS name	State	VPC ID	Availability Zones	Type	Date created
☐	ALB-kaif01	ALB-kaif01-1207662717.a...	Active	vpc-0cb56bce682200cc2	3 Availability Zones	application	November 12, 2024, 15:21 (UT...

< > ↻ ⚠ Not secure alb-kaif01-1207662717.ap-south-1.elb.amazonaws.com

ⓘ The extension upgrade requires additional permissions. Go to the extensions manager to accept the upgrade.

My name is Muhammad Kaif, ip-172-31-46-252.ap-south-1.compute.internal

ⓘ The extension upgrade requires additional permissions. Go to the extensions manager to accept the upgrade.

My name is Muhammad Kaif, ip-172-31-42-19.ap-south-1.compute.internal

The Application Load Balancer (ALB) is effectively directing traffic to both EC2 instances using a load balancing algorithm, likely round-robin. This ensures that incoming requests are evenly distributed across both instances, optimizing resource utilization and enhancing performance. Additionally, the ALB performs health checks to identify and remove unhealthy instances from the load balancing pool, guaranteeing that only healthy instances receive traffic. This mechanism ensures high availability and minimizes service disruptions.